

Best Practices für Databricks Apps

Empfehlungen für Sicherheit, Leistung und Plattform-Konformität bei Entwicklung und Betrieb von Apps.

Diese Seite fasst bewährte Vorgehensweisen zusammen, die Entwickler beim Bau und Betrieb von Databricks Apps beachten sollten. Sie helfen, Apps robuster, sicherer und performanter zu machen.

Allgemeine Best Practices

Datenverarbeitung: Native Dienste nutzen

Statt aufwendige Berechnungen direkt im App-Container durchzuführen, sollte man Databricks-native Dienste nutzen: **Databricks SQL** für Abfragen und Datensätze, **Lakeflow Jobs** für Batch-Verarbeitung (also die Verarbeitung großer Datenmengen im Hintergrund) und **Model Serving** für KI-Inferenz-Workloads (das Ausführen von KI-Modellen). App-Container sind für die Darstellung der Oberfläche optimiert, nicht für intensive Datenverarbeitung. Apps sollten unter realistischen Lastbedingungen getestet werden, um sicherzustellen, dass die Leistung den Anforderungen entspricht.

Sauberes Herunterfahren (Graceful Shutdown)

Apps müssen ein korrektes Beenden unterstützen. Erhält eine App das Signal `SIGTERM`, hat sie **15 Sekunden Zeit**, um sauber herunterzufahren, bevor das System sie zwangsweise mit `SIGKILL` beendet.

Betriebseinschränkungen

Apps laufen als nicht-privilegierte Nutzer und können keine Aktionen ausführen, die erhöhte Rechte erfordern. Insbesondere können **keine System-Pakete** über Paketmanager wie `apt-get`, `yum` oder `apk` installiert werden. Stattdessen sollten PyPI (für Python) bzw. npm (für Node.js) genutzt werden.

Netzwerkarchitektur verstehen

Databricks setzt eine plattformverwaltete Netzwerkarchitektur mit einem Reverse Proxy ein. Das bedeutet: Eine App kann sich nicht auf den Ursprung (Origin) eingehender Anfragen verlassen. Die Plattform übernimmt die TLS-Terminierung (also die Entschlüsselung der sicheren Verbindung), und Apps müssen HTTP/2 Cleartext (H2C) unterstützen. Eigene TLS-Implementierungen sind hier nicht sinnvoll.

Port- und Host-Konfiguration

Apps müssen auf `0.0.0.0` lauschen und den in der Umgebungsvariable `DATABRICKS_APP_PORT` angegebenen Port verwenden, damit das Routing korrekt funktioniert.

Startzeit optimieren

Die Initialisierungszeit des Containers sollte möglichst gering gehalten werden, um sogenannte "Cold-Start"-Verzögerungen (also die Wartezeit beim ersten Start) zu reduzieren. Blockierende Vorgänge beim Start – etwa aufwendige Installation von Abhängigkeiten oder externe API-Aufrufe – sollten vermieden werden. Ressourcenintensive Komponenten sollten nur bei tatsächlichem Bedarf geladen werden.

Logging-Strategie

Die Plattform erfasst die Ausgabe der Standard-Streams. Alle Logs sollten daher nach `stdout` und `stderr` geschrieben werden, um in der Databricks-Oberfläche sichtbar zu sein. Das Schreiben von Logs ins lokale Dateisystem sollte vermieden werden.

Fehlerbehandlung

Eine umfassende Ausnahmebehandlung (Exception Handling) sollte implementiert werden, um unerwartete Abstürze zu verhindern. Passende HTTP-Fehlercodes sollten zurückgegeben werden, wobei sensible Informationen (z. B. vollständige Stack-Traces) nicht offengelegt werden sollten.

Python-Abhängigkeitsverwaltung

Für alle Python-Anwendungen empfiehlt Databricks die Nutzung von `uv` (ein moderner, schneller Python-Paketmanager). Er bietet schnellere Installationen, reproduzierbare Builds über `uv.lock`-Dateien und flexible Auswahl der Python-Version. Wer `requirements.txt` verwendet, sollte exakte Versionsnummern angeben (Pinning), um Konsistenz über verschiedene Deployments hinweg sicherzustellen.

Eingabesicherheit

Eingehende Nutzereingaben sollten immer validiert und bereinigt werden, um Injection-Angriffe und fehlerhafte Daten zu verhindern – auch bei rein internen Anwendungen.

Caching-Strategie

Für aufwendige Operationen wie Abfrageergebnisse oder API-Antworten lohnt sich In-Memory-Caching (Zwischenspeicherung im Arbeitsspeicher), um Latenz zu reduzieren und redundante Verarbeitung zu vermeiden. Werkzeuge wie `functools.lru_cache` oder `cachetools` eignen sich dafür, wobei der Gültigkeitsbereich (Scope) in Umgebungen mit mehreren gleichzeitigen Nutzern sorgfältig verwaltet werden muss.

Asynchrone Muster für lange Vorgänge

Synchrone Anfragen, die auf den Abschluss eines langwierigen Vorgangs warten, sollten vermieden werden, da sie in ein Timeout laufen können. Stattdessen sollte der Vorgang gestartet und der Status danach periodisch über einen eigenen Status-Endpunkt abgefragt werden.

Sicherheits-Best-Practices

Berechtigungsmodell

Es gilt das Prinzip der minimalen Rechtevergabe (Least Privilege): Nur die tatsächlich benötigten Berechtigungen sollten vergeben werden. `CAN USE` sollte gegenüber `CAN MANAGE` bevorzugt werden, außer wenn vollständige Kontrolle wirklich erforderlich ist.

Wahl der Authentifizierungsmethode

Service Principals eignen sich am besten, wenn alle Nutzer der App denselben Zugriff auf Ressourcen und Daten benötigen. Nutzer-Authentifizierung sollte nur in Workspaces mit vertrauenswürdigen Entwicklern und im Peer-Review geprüfem Code eingesetzt werden – vor allem dann, wenn die App tatsächlich nutzerspezifische Berechtigungen durchsetzen muss.

Verwaltung von Service Principals

Für jede App sollte ein eigenes, dediziertes Service Principal verwendet werden, statt Zugangsdaten über mehrere Apps oder Nutzer hinweg zu teilen. Es sollten nur minimale

Berechtigungen wie `CAN USE` oder `CAN QUERY` vergeben werden. Zugangsdaten sollten rotiert werden, wenn Teammitglieder mit App-Erstellungsrechten das Unternehmen verlassen.

Trennung der Umgebungen

Entwicklungs-, Staging- und Produktions-Apps sollten in unterschiedlichen Workspaces liegen, um versehentlichen Zugriff auf Produktivdaten während des Testens zu vermeiden.

Datenzugriffsmuster

Apps sollten über passende Compute-Ressourcen auf Daten zugreifen, statt direkte Verbindungen aufzubauen: SQL-Warehouses für Abfragen, Model Serving für KI-Inferenz und Lakeflow Jobs für Batch-Verarbeitung.

Verwaltung von Secrets

Geheime Werte (Secrets) sollten niemals im Klartext in Umgebungsvariablen erscheinen. Stattdessen sollte die Option `valueFrom` genutzt werden, verbunden mit regelmäßiger Rotation der Zugangsdaten, insbesondere bei Rollenwechseln im Team.

Begrenzung des Autorisierungsumfangs

Bei Nutzer-Autorisierung sollten nur die tatsächlich notwendigen Berechtigungsbereiche (Scopes) angefordert werden. Zusätzlich sollten strukturierte Audit-Protokolle für alle Nutzeraktionen geführt werden.

Netzwerkzugriffskontrolle

Ausgehende Netzwerkverbindungen sollten auf die tatsächlich benötigten Domains beschränkt werden, etwa Paket-Repositories und notwendige externe APIs. Die Konfiguration lässt sich über einen Dry-Run-Modus und Ablehnungs-Logs (Denial Logs) validieren.

Schutz vor SQL-Injection

SQL-Abfragen sollten parametrisiert werden, um Injection-Angriffe zu verhindern. Zusätzlich gelten die üblichen Grundsätze sicherer Entwicklung: Eingabevalidierung und umfassende Fehlerbehandlung.

Sicherheitsüberwachung

Audit-Logs sollten regelmäßig auf ungewöhnliche Zugriffsmuster oder unautorisierte Aktivitäten überprüft werden. Für kritische Sicherheitsereignisse sollten Alarme eingerichtet werden.

Zusammengefasst: Die wichtigsten Leitplanken sind – native Databricks-Dienste für schwere Datenverarbeitung nutzen, App-Container schlank und zustandslos halten, konsequent nach dem Prinzip der minimalen Rechte arbeiten, und Secrets sowie Netzwerkzugriffe eng kontrollieren.

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/best-practices>
Stand: 2026-08-06